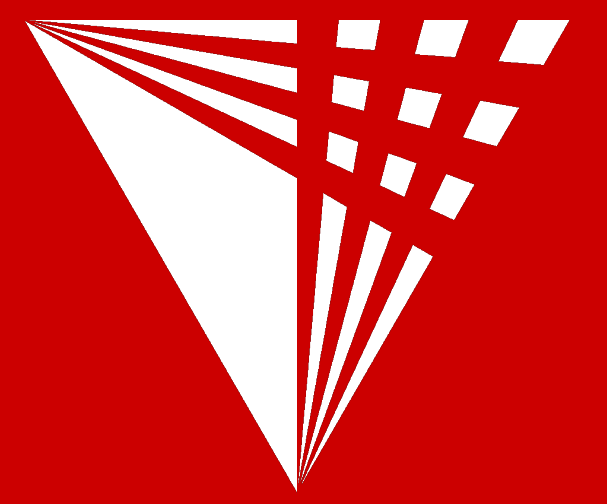


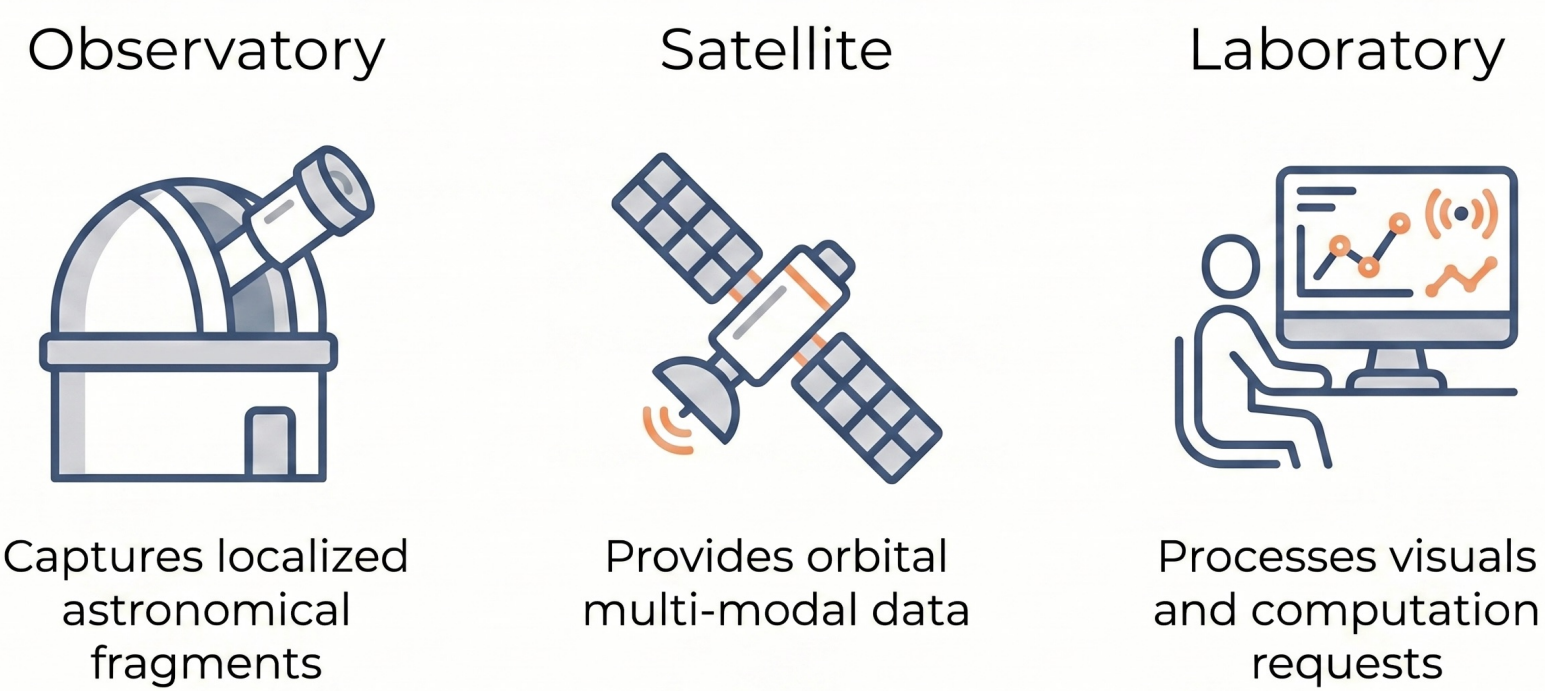


Motivation

Scientific discovery increasingly relies on collaboration between diverse institutions, such as telescope observatories, satellites, and laboratories, which each observe only a fragment of the underlying phenomenon.

Most scientific data is decentralized. How do we do inferencing?

Modality: A *data feature* that can be in any form (e.g. numerical data, images, videos).



As such, these institutions use diverse modes of data for inferencing through complex calculations. However, collecting every data modality or feature relevant to the calculation is not realistically possible for individual institutions. Thus, each node v has certain modes of data M_v for certain timestamps T_v .

- **Example:** A telescope observatory can collect specific types of data about the sky it is pointed towards (and only at night). It would not have the means to collect gravitational wave data.
- In practice, this creates **partial, timestamped, multimodal observations** distributed over many scientific silos.

Problem Setting

- We study **private, distributed inference** when parties hold **partial multimodal observations** and cannot directly share raw data.
- Each worker node locally stores a subset of modalities over local timestamps and submits a request whenever a fixed scientific calculation cannot be completed from its local view.
- We now treat requests as arriving in **batches**, not one at a time.
- The orchestrator must therefore decide not only *who can serve each missing modality*, but also *which nodes should be reused across the batch*.

For a request batch R , the optimization trades off:

- **Communication cost:** modality weight $\times$ hop distance
- **Activation cost:** the cost of opening a new node for the batch

System Architecture

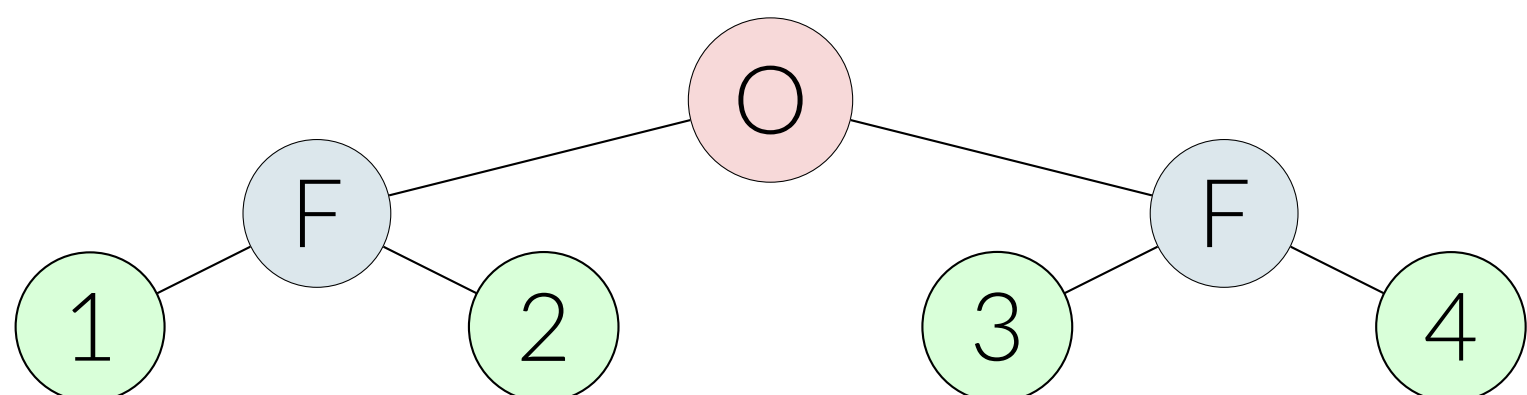


Figure 1. Balanced-tree topology used in the experiments. The trusted **orchestrator** coordinates batch request fulfillment over worker silos through intermediate forwarders.

- The **orchestrator** is the trusted root service provider.
- **Forwarder nodes** relay traffic between the orchestrator and worker silos.
- **Worker nodes** hold raw observations locally and only contribute the modalities needed to complete requests.
- We study the problem on balanced-tree topologies, with the main modeling focus placed on batch-coupled routing over decentralized silos.

Single-Request Toy Model

We begin with the simplest useful routing problem: one request at timestamp t ,

$$r(t) = (t, M \setminus M_a),$$

where M is the full modality set needed for the scientific calculation and $M_a \subseteq M$ is the subset already available at the requesting node. The missing set is therefore $M_r = M \setminus M_a$.

For each missing modality $m \in M_r$, the orchestrator chooses one provider node $v_m \in V$. The toy objective is:

$$\min \sum_{m \in M_r} h_{v_m} \text{weight}_m,$$

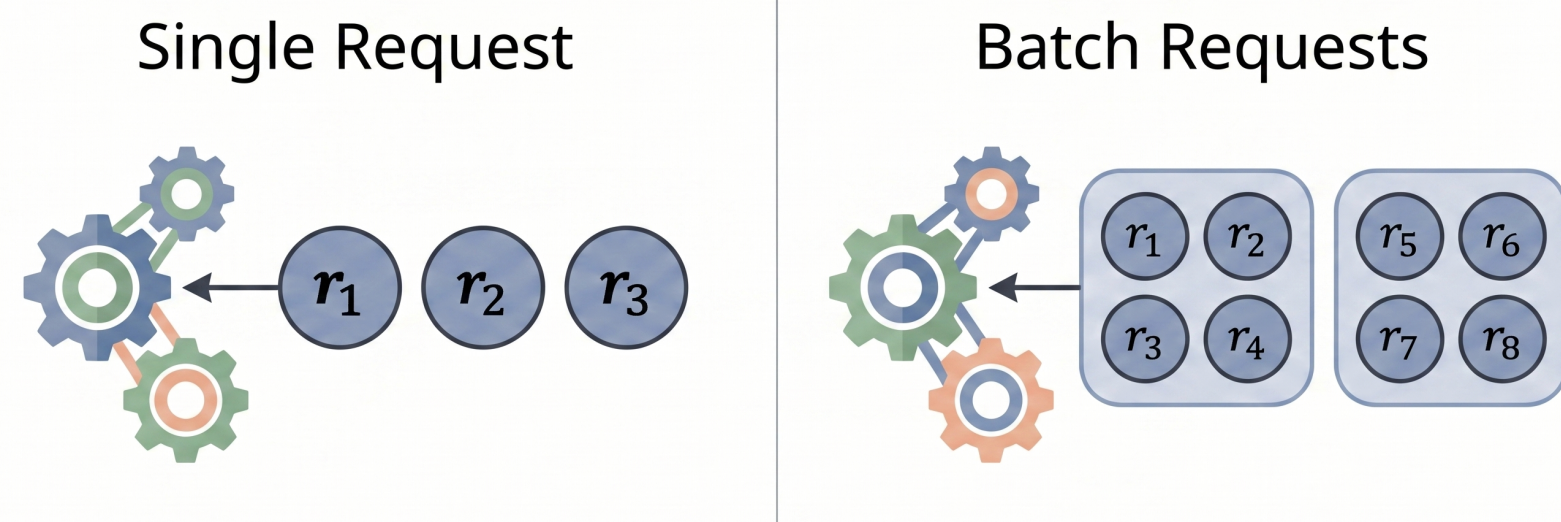
where h_{v_m} is the hop distance from the orchestrator to provider v_m and weight_m is the transfer size or cost of modality m .

The toy model is useful for intuition, but it is still an **independent assignment** view: each missing modality is routed separately. The combinatorial difficulty begins once node selection is coupled across multiple required modality-timestamp pairs, and becomes richer still when many requests are optimized together.

Study Goal

Can we **choose provider nodes** for a batch of requests while minimizing objective value and preserving practical runtime?

Expansion to Batch



Same routing problem, but now with **shared node reuse across many requests**.

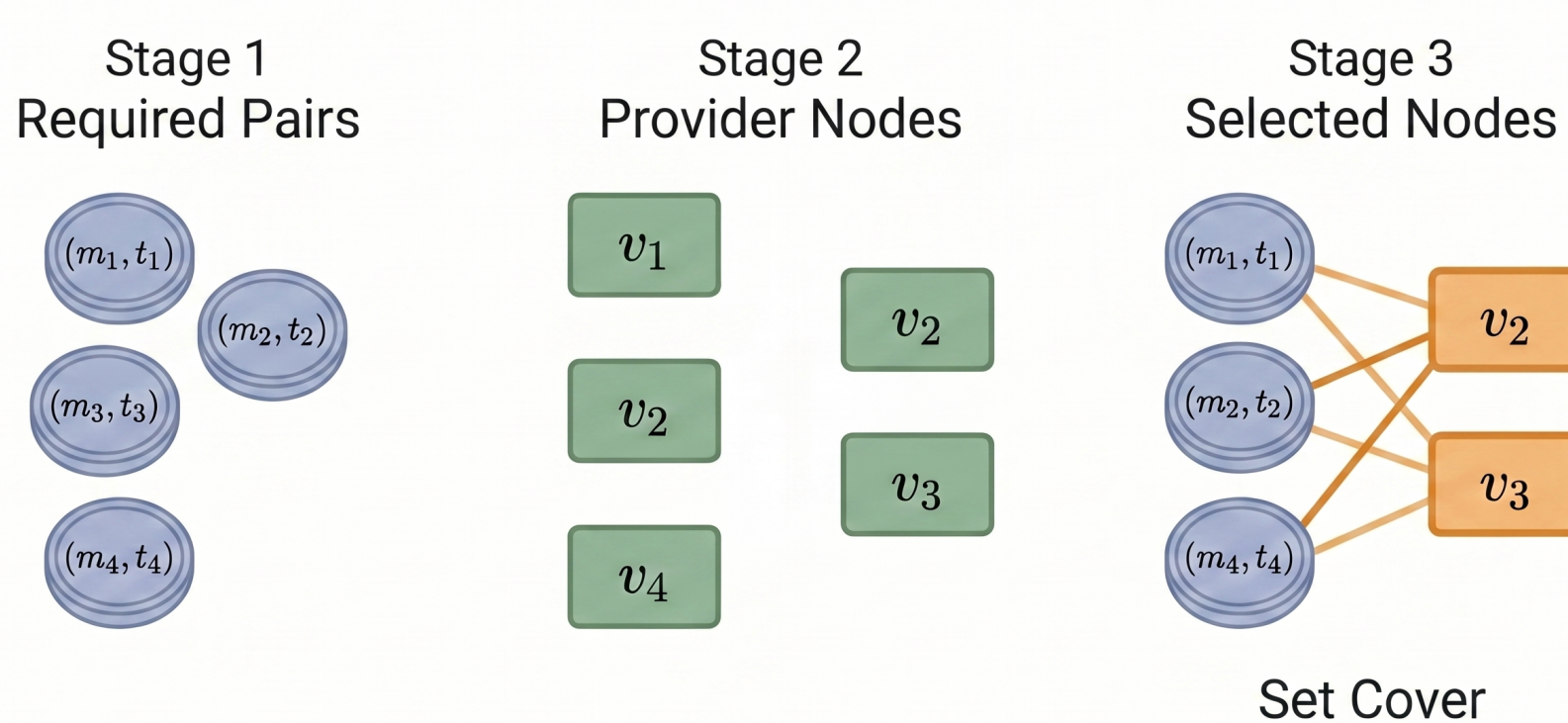
$$\min \underbrace{\sum_{r \in R} \sum_{m \in M_r} \sum_{v \in V} x_{rvm} h_v \text{weight}_m}_{\text{communication}} + \underbrace{\sum_{v \in V} y_v \lambda h_v}_{\text{activation / reuse penalty}}$$

- $x_{rvm} = 1$: node v serves modality m for request r
- $y_v = 1$: node v is activated in the batch
- h_v : hop distance weight_m : transfer weight λ : reuse penalty

A slightly worse local route can still be better globally if it avoids activating more nodes.

Why the Problem is NP-hard

Routing Complexity: The Set Cover Reduction



For one request r , define the required universe

$$U_r = \{(m, t) : (m, t) \text{ is required by } r\}.$$

Each provider node v contributes a subset

$$S_v = \{(m, t) \in U_r : v \text{ stores modality } m \text{ at timestamp } t\}.$$

- required pairs $(m, t) \rightarrow$ items
- provider nodes $v \rightarrow$ sets
- selected nodes $\rightarrow$ a set cover

Choosing the minimum-cost nodes whose union covers U_r is exactly **Weighted Set Cover**. Batching extends this already-hard node-cover problem through shared activation variables.

Integer Linear Program (ILP) Formulation

The ILP is the **exact** batch baseline. Here, $a_{rvm} = 1$ only when node v can serve modality m for request r .

$$\begin{aligned} \min_{x, y} \quad & \sum_{r \in R} \sum_{m \in M_r} \sum_{v \in V} x_{rvm} h_v \text{weight}_m + \sum_{v \in V} y_v \lambda h_v \\ \text{s.t.} \quad & \sum_{v \in V} x_{rvm} = 1 \quad \forall r \in R, \forall m \in M_r \\ & x_{rvm} \leq a_{rvm} \quad \forall r \in R, \forall m \in M_r, \forall v \in V \\ & x_{rvm} \leq y_v \quad \forall r \in R, \forall m \in M_r, \forall v \in V \\ & x_{rvm} \in \{0, 1\}, \quad y_v \in \{0, 1\} \end{aligned}$$

- $\sum_v x_{rvm} = 1$: every requested modality is assigned once
- $x_{rvm} \leq a_{rvm}$: only available data can be used
- $x_{rvm} \leq y_v$: using a node activates it

Exact objective optimization, but slower than the proposed heuristics.

Algorithms and Study Design

- **ILP:** exact batch-coupled baseline.
- **Proposed:** penalty-aware batch heuristic that scores inactive nodes using the same reuse penalty as the ILP objective.
- **Proposed Fast:** lookup-accelerated variant of the proposed heuristic.
- **Random / Random Fast:** reuse-aware stochastic baselines.

Experimental controls

- Balanced-tree branching factors: $\{2, 10\}$
- Node counts: $\{10, 20, 50, 100, 200\}$
- Requests per node: $\{10, 100\}$
- Batch sizes: $\{8, 16, 32\}$
- Data uniformity: $\{1, 100, 100000\}$

Objective Value Per Batch

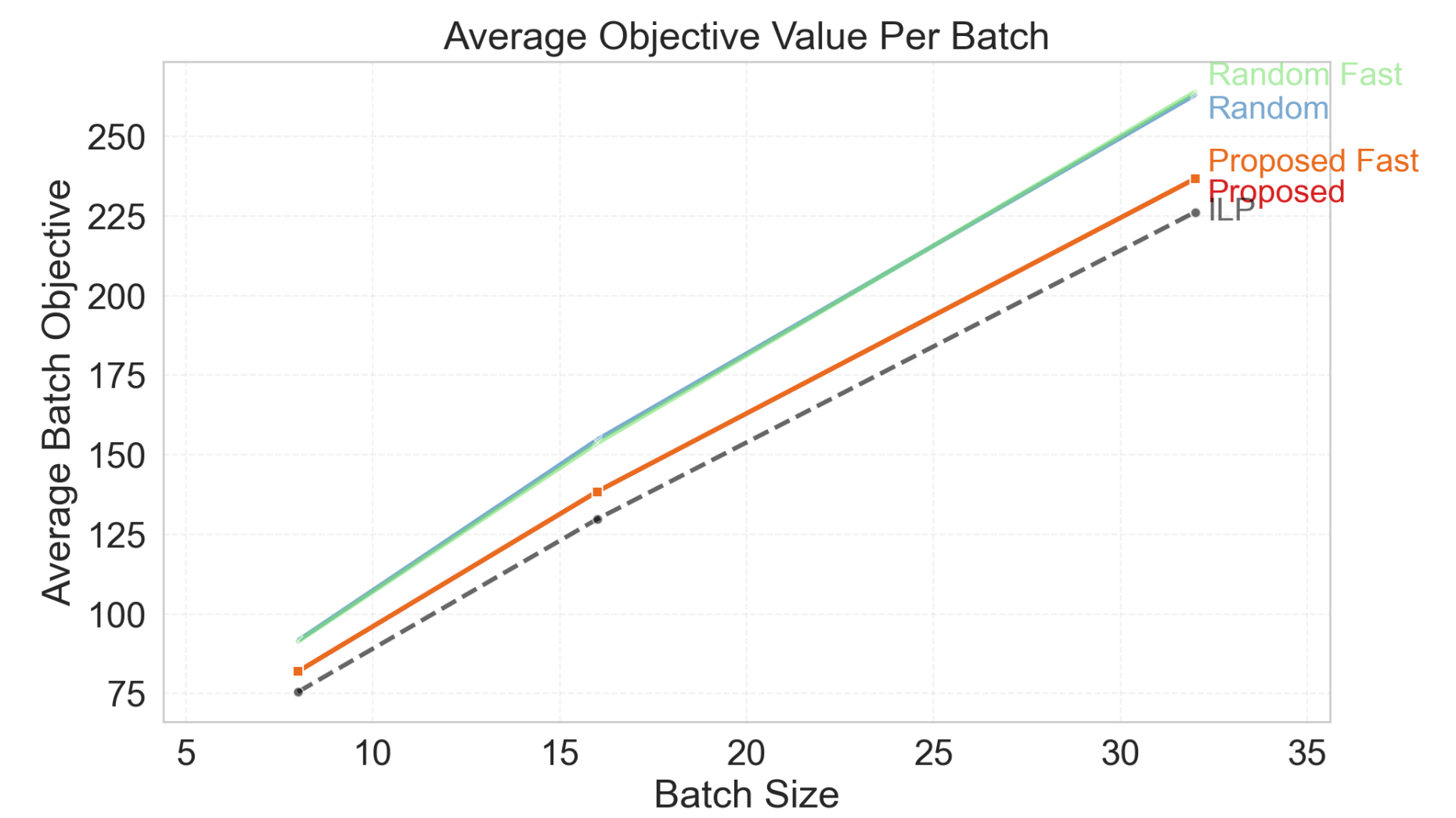


Figure 2. Average batch objective value. This is a simple comparison for the batch formulation because it includes both communication and activation cost.

Average Runtime Per Batch

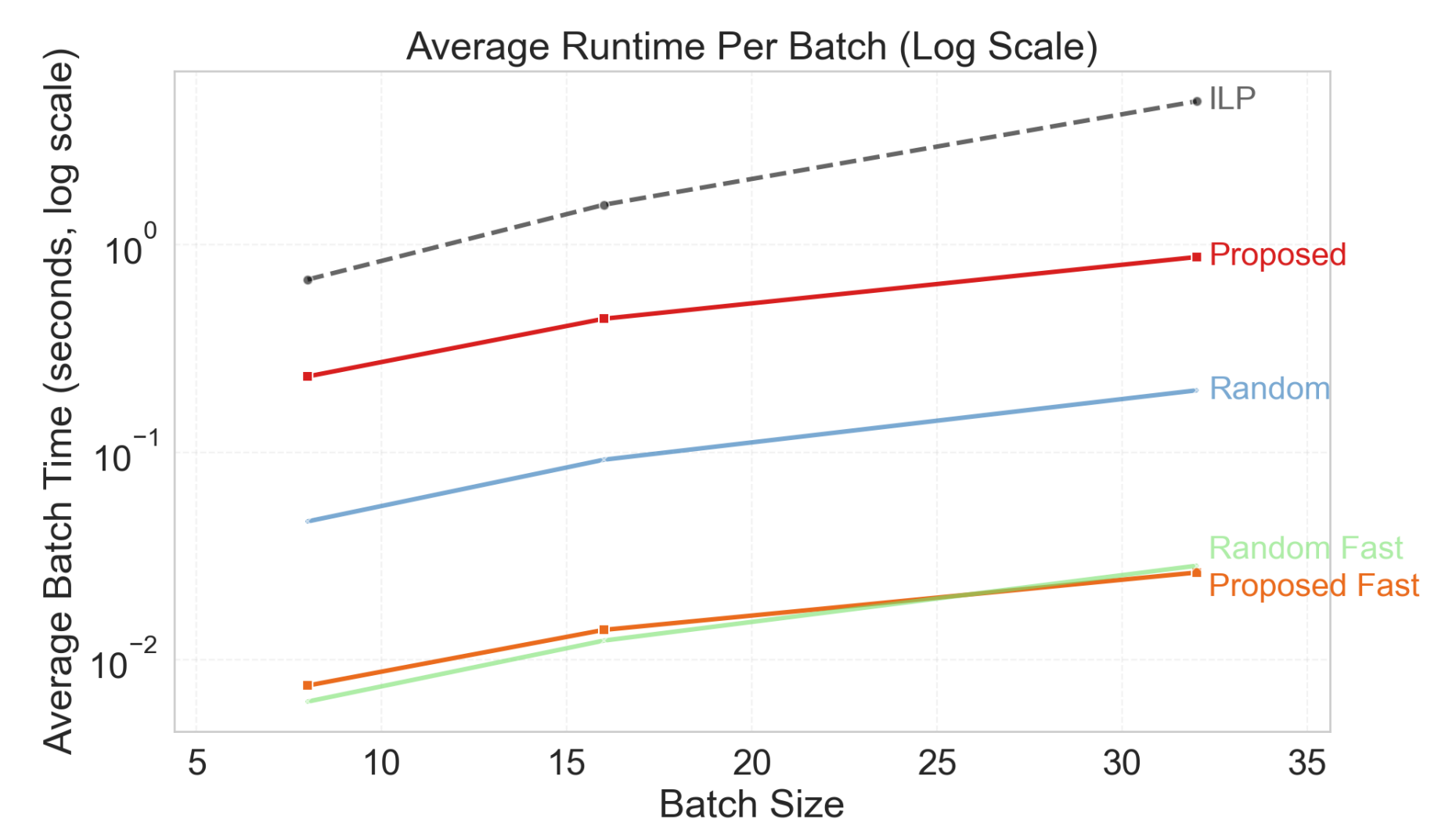


Figure 3. Average orchestration time per batch. The goal is to tolerate slightly higher per-request communication when node reuse substantially lowers practical orchestration effort.

Average Activated Nodes Per Batch

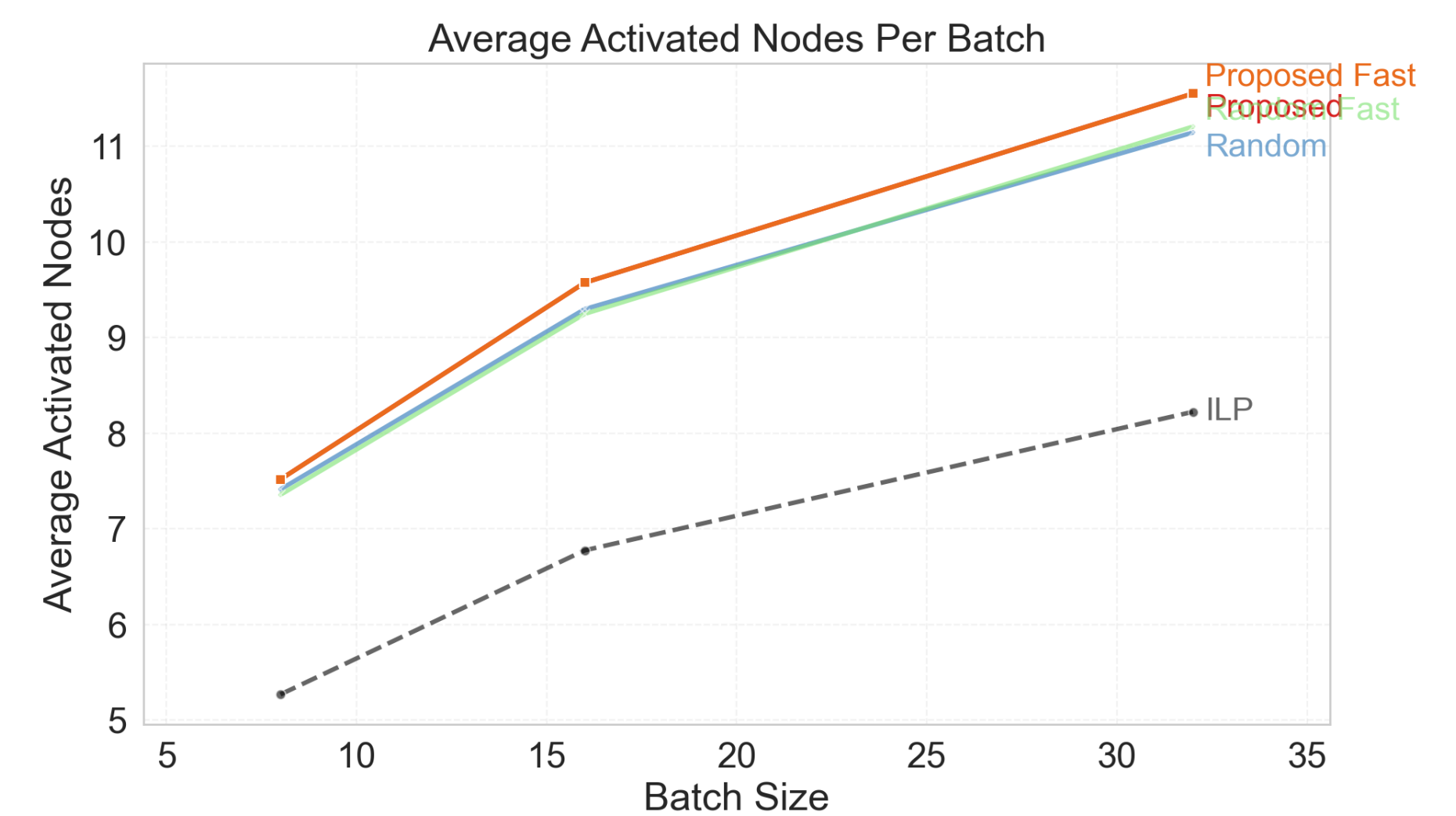


Figure 4. Average number of distinct nodes touched per batch. Lower values indicate stronger reuse across requests.

Results Snapshot

Alg.	Obj/B (↓)	Req (↓)	Time/B (↓)	Act/B (↓)
ILP (<i>baseline</i>)	143.84	8.87	2.389	6.76
Proposed	152.36	8.80	0.515	9.55
Prop. Fast	152.36	8.80	0.016	9.55
Random	169.80	10.11	0.112	9.29
Rand. Fast	169.67	10.09	0.016	9.27

Req = average cost per request, **Obj/B** = average objective per batch, **Time/B** = average time per batch, **Act/B** = average activated nodes per batch. All methods achieved 100% success with 0 mean failures.

Key Findings

- All five methods achieved **100% success** with **0 mean failures**.
- ILP achieved the best batch objective: **143.84** vs. **152.36** for **proposed_fast** and **169.67** for **random_select_fast**.
- **proposed_fast** was about **150× faster** than ILP per batch: **0.016** vs. **2.389**.
- ILP used the fewest activated nodes per batch: **6.76** vs. **9.55** for **proposed_fast**, a gap of about **41%**.

Takeaways and Next Steps

- The proposed methods are already close to ILP on communication cost, but still lag on node reuse.
- The clearest discrepancy in the results is the number of activated nodes per batch.
- A strong next step is designing heuristics that preserve the speed of **proposed_fast** while moving closer to ILP-level node consolidation.